Claude Code cheat sheet

Ten rules you can follow blindly. Same ticket, recorded twice: these rules cut the cost by 57%.

The meter: every message re-sends the whole conversation, including every file read and every log pasted. Cost depends on context size, not message length. A “thanks” on a 130k session re-reads 130k.

1 One task, one session

Task changes, run `/clear`. Side questions go in a separate chat.

2 Plan before you build

Multi-file change: 10 to 20 line spec with out-of-scope, then plan mode (`Shift+Tab`), then approve.

3 Pick the right model

Sonnet by default, Opus to plan or debug, `/model opusplan` for both. Switch only right after `/clear`.

4 Point, don't make it search

Name files with `@path` and an existing pattern to copy.

5 Keep CLAUDE.md short

Commands, hard rules, a link to the architecture map, guardrails. Sent on every message.

6 Two strikes, then rewind

Corrected twice? `Esc Esc` or `/rewind`, then rewrite the prompt with what was missing.

7 Facts, not feelings

“I did X, expected Y, got Z.” Collect issues and send them in one message.

8 Trim what you paste

Failing test name and the error lines, not the whole log. “Report only failures.”

9 Hand off, don't drag on

At a milestone: `/handoff`, `/clear`, “Read @.claude/handoff.md and continue.”

10 No unrequested docs

“Do not create markdown files unless asked.” Turn off MCP servers you aren't using.

Commands

`/context`	See the meter
`/clear`	Fresh session
`/compact <keep…>`	Shrink, saying what to keep
`/rewind`, `Esc Esc`	Undo a bad turn
`/model opusplan`	Opus plans, Sonnet builds
`Shift+Tab`	Plan mode
`/cost`, `/usage`	Measure (run before `/clear`)
`/mcp`	Turn servers off
`! cmd`	Run it yourself

The workflow

1. Spec in `docs/specs/`
2. `/clear`, `/model opusplan`, plan mode
3. Review plan, approve (clear context)
4. Build, run checks, report failures only
5. Check yourself, send all fixes at once
6. Ship: commit, push, PR from the spec

Spec template

```
# Spec: <name> (<ticket>)
Goal: <what changes for the user>
- Data: <schema, migrations>
- Logic: <where; pattern to follow>
- API: <endpoints, errors>
- UI: <where; empty/error states>
- Tests: <cases>
- Out of scope: <what not to do>
- Done when: <checks; PR opened>
```

Before you send a prompt

Purpose	What, and for whom?
Context	Which files and patterns?
Specifics	What would it have to guess?
Out of scope	What must it not do?
Done when	How will you both know?

Full guide: guide/guide.pdf. Commands checked on Claude Code 2.1.287; run /help on your version.